

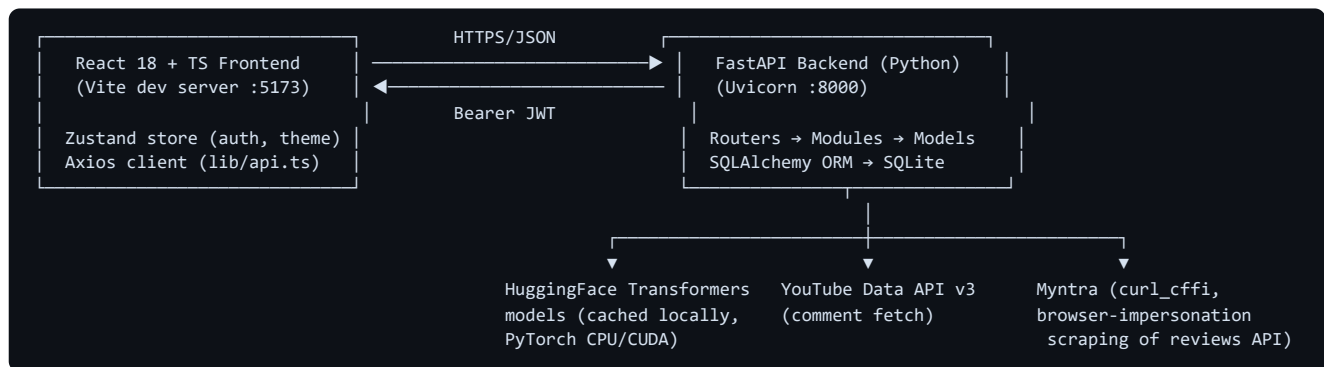
Codebase Gist — Sentiment Intelligence Platform

What this project is

A full-stack, multilingual NLP analytics platform (Final Year Project). It takes raw text — typed directly, uploaded as a CSV, or scraped from a YouTube/Myntra URL — and runs it through an 8-stage AI pipeline that produces sentiment, sarcasm, emotion, tone, intent, a human-readable interpretation, and a suggested reply. Results are persisted to SQLite, aggregated into a live analytics dashboard, exportable as branded PDF reports, and improvable over time through an admin-driven correction/retraining loop.

Stack: FastAPI + SQLAlchemy + SQLite backend, HuggingFace Transformers/PyTorch models, React 18 + TypeScript + Vite + Tailwind frontend, Zustand for state, JWT-based auth with role-based (`user` / `admin`) access control.

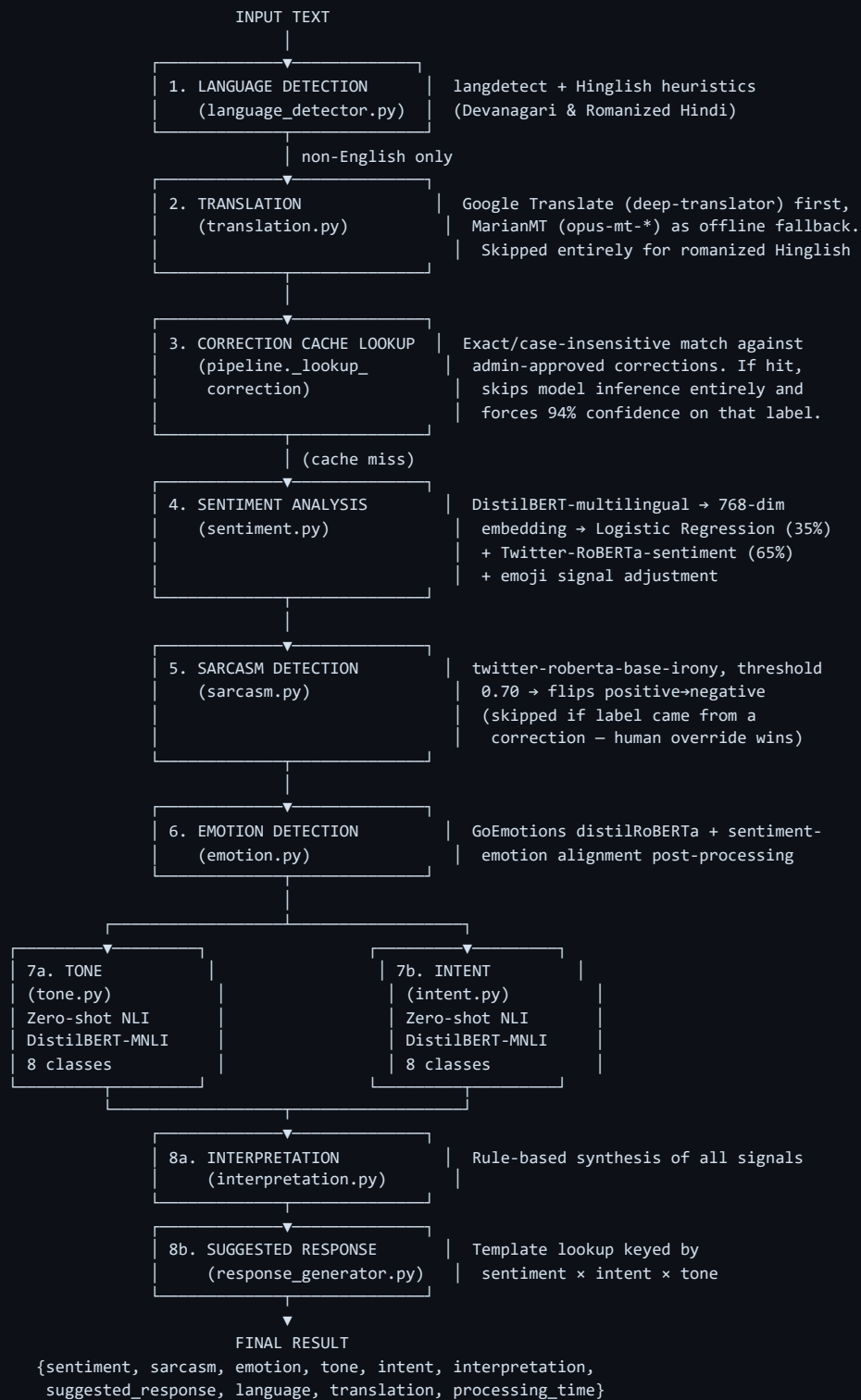
1. System Architecture



Request flow for a typical analysis: Frontend → `POST /api/v1/analyze` (JWT in header) → `routers/analysis.py` → `modules/pipeline.analyze_text()` runs the 8-step AI pipeline → result persisted as an `AnalysisRecord` row → JSON returned to frontend → rendered as per-signal cards on `SingleAnalysis.tsx`.

2. The Core AI Pipeline (`modules/pipeline.py::analyze_text`)

Every piece of text — typed, CSV row, or scraped comment — passes through the same pipeline:

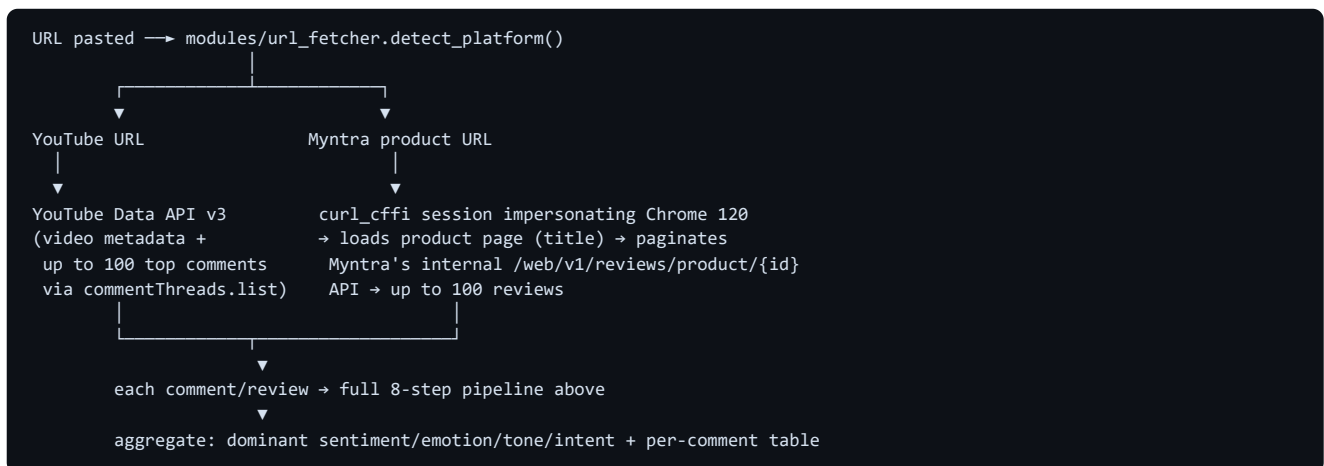


AI models used

Stage	Model	Notes
Sentiment	<code>distilbert-base-multilingual-cased</code> (embeddings) + Logistic Regression	LR trained in-memory at startup on seed examples; 35% ensemble weight
Sentiment	<code>cardiffnlp/twitter-roberta-base-sentiment-latest</code>	Tweet-native, 65% ensemble weight
Sarcasm	<code>cardiffnlp/twitter-roberta-base-irony</code>	Threshold 0.70, flips positive→negative
Emotion	<code>j-hartmann/emotion-english-distilroberta-base</code> (GoEmotions)	Aligned against sentiment post-hoc
Tone + Intent	<code>typeform/distilbert-base-uncased-mnli</code>	Zero-shot NLI, shared model, two label sets
Translation	Google Translate (<code>deep-translator</code> , no API key) → Helsinki-NLP <code>opus-mt-*</code> (MarianMT) fallback	Google Translate tried first for any language; MarianMT per-language models used only if it fails

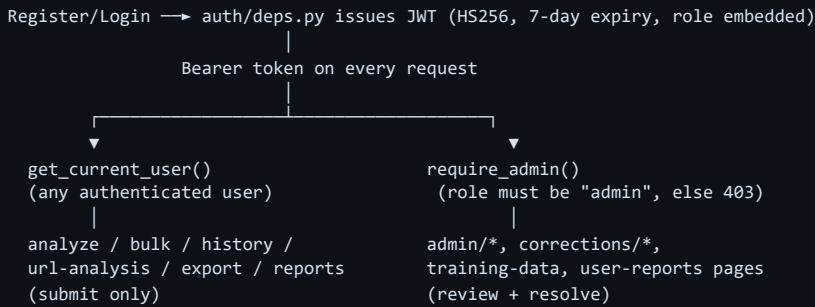
Supported languages: English, Hindi, Bengali, Assamese, Hinglish (Devanagari + Roman), Urdu, Tamil, Telugu, Gujarati, French, German, Spanish, Italian, Portuguese, Russian, Chinese, Japanese, Korean, Arabic.

3. URL Analysis Flow (YouTube / Myntra)



Myntra has no public reviews API, so `fetch_myntra_reviews()` in `modules/url_fetcher.py` impersonates a real Chrome browser via `curl_cffi` (TLS/JA3 fingerprint spoofing) to avoid being blocked, extracts the product ID from the URL path, scrapes the `<title>` tag for the product name, then paginates Myntra's internal reviews JSON endpoint directly.

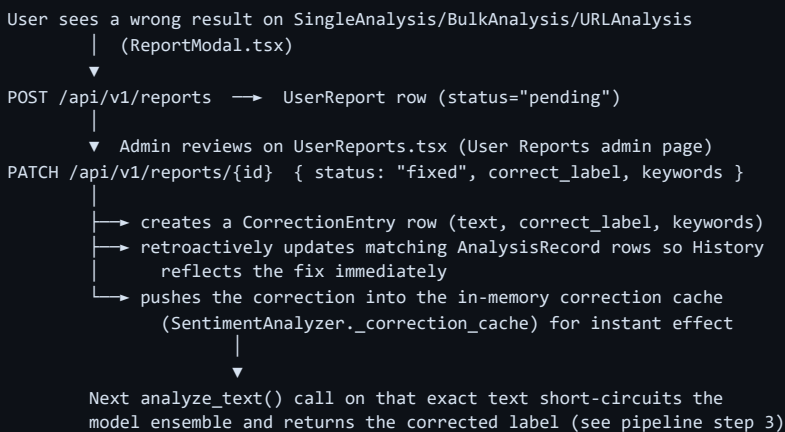
4. Auth & Role-Based Access



- Passwords hashed with bcrypt (`passlib`).
- `is_active` flag lets admins restrict a user's account without deleting it.
- Frontend guards routes client-side (`RequireAuth` , `RequireAdmin` in `App.tsx`) in addition to backend 401/403 enforcement.

5. Continuous-Improvement / Feedback Loop

This is the platform's self-correction mechanism — the most distinctive architectural piece beyond the base pipeline:



Additional admin tooling around this loop (`routers/corrections.py` , `TrainingData.tsx`): - `GET /corrections/stats` — label breakdown + top correction keywords, `needs_retrain` flips true at 20 pending corrections (`RETRAIN_THRESHOLD`). - `POST /corrections/fetch-online` — pulls additional labeled samples to enrich the seed training set. - `POST /corrections/retrain` — rebuilds the Logistic Regression head using all accumulated corrections. - Corrections persist in SQLite and are **reloaded into the in-memory cache on every server restart** (see `main.py` lifespan handler), so fixes are never lost.

6. Admin Panel (`AdminOverview.tsx` , `routers/admin.py`)

- **User management:** list all users, toggle `is_active` (restrict/unrestrict), delete users (cannot self-restrict/self-delete).
- **Storage gauge:** SQLite file size vs. a 500 MB soft cap, plus row counts for analyses/corrections/reports.
- **Data lifecycle:** `DELETE /admin/clear-analysis` (wipe analysis history) and `DELETE /admin/clear-all` (wipe analyses + corrections + reports, users kept).

7. Directory Structure

```

FinalYearProject/
├── backend/
│   ├── main.py                # FastAPI app, CORS, router mounts, lifespan startup
│   ├── config.py              # Pydantic settings (.env-driven)
│   ├── auth/
│   │   └── deps.py            # JWT issue/verify, bcrypt hashing, get_current_user/require_admin
│   ├── modules/
│   │   ├── pipeline.py        # AI/ML pipeline + integrations
│   │   ├── language_detector.py # Orchestration - analyze_text(), correction lookup, emotion alignment
│   │   ├── translation.py     # langdetect + Hinglish heuristics
│   │   ├── sentiment.py       # translate_to_english(): Google Translate primary, MarianMT fallback
│   │   ├── sarcasm.py         # BERT+LR ensemble, correction cache, retraining, online data fetch
│   │   ├── emotion.py         # twitter-roberta-base-irony detector
│   │   ├── tone.py           # GoEmotions distilRoBERTa
│   │   ├── intent.py          # Zero-shot NLI tone classifier
│   │   ├── interpretation.py  # Zero-shot NLI intent classifier
│   │   ├── response_generator.py # Rule-based contextual synthesis
│   │   ├── analytics.py       # Template response suggestions
│   │   ├── url_fetcher.py     # Dashboard aggregation + insights
│   │   └── pdf_generator.py   # YouTube Data API + Myntra reviews scraper (curl_cffi)
│   │   # PDF export (fpdf2)
│   ├── routers/
│   │   ├── analysis.py        # POST /analyze
│   │   ├── bulk.py            # POST /bulk-analyze, POST /upload-csv
│   │   ├── analytics.py       # GET /analytics
│   │   ├── history.py         # GET/DELETE /history
│   │   ├── url_analysis.py    # POST /analyze-url
│   │   ├── export.py          # POST /export/pdf/*
│   │   ├── auth.py            # POST /auth/register, /auth/login, GET /auth/me
│   │   ├── corrections.py     # Correction CRUD + retrain + online-data fetch (admin only)
│   │   ├── reports.py         # User-submitted mis-classification reports + admin review
│   │   └── admin.py           # User management, storage stats, data wipe (admin only)
│   ├── database/
│   │   ├── connection.py      # SQLAlchemy engine + session factory + init_db()
│   │   └── models.py          # User, UserReport, CorrectionEntry, AnalysisRecord
│   ├── schemas/
│   │   # Pydantic request/response models (mirrors routers/)
│   └── requirements.txt
├── frontend/
│   └── src/
│       ├── App.tsx            # Routes, RequireAuth/RequireAdmin guards
│       ├── pages/
│       │   ├── Login.tsx      # Register/login
│       │   ├── Dashboard.tsx  # KPI cards, charts, trend, keyword cloud
│       │   ├── SingleAnalysis.tsx # Single text input + full result cards
│       │   ├── BulkAnalysis.tsx # Bulk text / CSV upload + results table
│       │   ├── URLAnalysis.tsx # YouTube / Myntra URL input + results
│       │   ├── History.tsx    # Paginated searchable history
│       │   ├── Settings.tsx   # Platform/theme settings
│       │   ├── AdminOverview.tsx # Admin dashboard (users, storage) [admin only]
│       │   ├── TrainingData.tsx # Correction stats + retrain trigger [admin only]
│       │   └── UserReports.tsx # Review/resolve user-submitted reports [admin only]
│       ├── components/
│       │   ├── Layout.tsx, Sidebar.tsx, BackendStatus.tsx
│       │   ├── ReportModal.tsx # "Report incorrect result" dialog (any user)
│       │   ├── CorrectionPanel.tsx # Admin correction entry UI
│       │   ├── analysis/
│       │   │   └── ui/
│       │   │       # Per-signal result cards (Sentiment, Emotion, Tone, Intent, ...)
│       │   │       # Button, Card, Input, Badge, Toast, Select, Progress, Skeleton
│       │   ├── lib/
│       │   │   ├── api.ts      # Axios client, attaches JWT, base URL from VITE_API_URL
│       │   │   ├── themes.ts   # Theme definitions
│       │   │   └── utils.ts
│       ├── store/index.ts      # Zustand: auth (token/user/logout), theme
│       └── types/index.ts      # Shared TypeScript interfaces

```

8. Data Model (SQLite via SQLAlchemy)

Table	Purpose	Key columns
<code>users</code>	Accounts	<code>username</code> , <code>email</code> , <code>password_hash</code> , <code>role</code> (<code>user</code> / <code>admin</code>), <code>is_active</code>
<code>analysis_records</code>	Every analysis result (single + bulk)	full signal breakdown (sentiment/emotion/tone/intent + scores), <code>mode</code> , <code>batch_id</code>
<code>user_reports</code>	User-flagged mis-classifications	<code>text</code> , <code>model_label</code> , <code>user_note</code> , <code>status</code> (pending/reviewed/fixed), <code>reported_by</code>
<code>correction_entries</code>	Admin-approved label corrections	<code>text</code> , <code>correct_label</code> , <code>model_label</code> , <code>keywords</code> — feeds the in-memory correction cache

9. API Reference (base: `/api/v1`)

Method	Endpoint	Auth	Description
POST	<code>/auth/register</code>	—	Create account, returns JWT
POST	<code>/auth/login</code>	—	Returns JWT (403 if <code>is_active=false</code>)
GET	<code>/auth/me</code>	user	Current user profile
POST	<code>/analyze</code>	user	Single text analysis
POST	<code>/bulk-analyze</code>	user	Up to 100 texts
POST	<code>/upload-csv</code>	user	Parse CSV → extract text column
POST	<code>/analyze-url</code>	user	YouTube/Myntra comment/review scrape + analyze
GET	<code>/analytics</code>	user	Dashboard KPIs, trends, keyword cloud
GET/DELETE	<code>/history</code> , <code>/history/{id}</code>	user	Paginated history, delete record(s)
POST	<code>/export/pdf/{single,bulk,url}</code>	user	Branded PDF report
POST	<code>/reports</code>	optional	Submit a mis-classification report
GET/PATCH/DELETE	<code>/reports*</code>	admin	Review, resolve, delete reports
GET/POST/DELETE	<code>/corrections*</code>	admin	Manage correction entries
POST	<code>/corrections/retrain</code>	admin	Rebuild LR head from corrections
POST	<code>/corrections/fetch-online</code>	admin	Pull additional labeled seed data
GET	<code>/admin/stats</code>	admin	User counts, storage gauge, record counts
PATCH	<code>/admin/users/{id}/restrict</code>	admin	Toggle account active/restricted
DELETE	<code>/admin/users/{id}</code>	admin	Delete a user
DELETE	<code>/admin/clear-analysis</code> , <code>/admin/clear-all</code>	admin	Wipe stored data
GET	<code>/health</code>	—	Health check

10. Environment / Config (`backend/config.py` , `.env`)

```
DATABASE_URL=sqlite:///./sentiment_platform.db
MODEL_CACHE_DIR=./model_cache
DEVICE=auto          # auto | cuda | cpu
JWT_SECRET=<change in production>
JWT_ALGORITHM=HS256
JWT_EXPIRE_DAYS=7
CORS_ORIGINS=[...]
YOUTUBE_API_KEY=...
INSTAGRAM_SESSION_ID=... # preferred over username/password (bypasses bot detection)
```

Frontend: `VITE_API_URL=http://localhost:8000/api/v1` (`frontend/.env`).

11. Tech Stack — What Each Piece Does and Why

Backend runtime

Technology	Role in this project	Why it was chosen
Python 3.11	Language the entire backend and ML pipeline is written in	Required by the HuggingFace/PyTorch ecosystem; best library support for NLP
FastAPI 0.115	Web framework — defines every <code>/api/v1/*</code> route, request/response validation, auto-generated docs at <code>/docs</code>	Async-native, integrates natively with Pydantic for typed request/response models, minimal boilerplate compared to Flask/Django
Uvicorn	ASGI server that actually runs the FastAPI app (<code>uvicorn main:app</code>)	The standard production-grade server for async Python web apps
Pydantic 2 / pydantic-settings	Validates every request body against a schema (<code>schemas/</code>) and loads <code>backend/.env</code> into a typed <code>Settings</code> object (<code>config.py</code>)	Catches malformed input before it reaches business logic; typed config avoids <code>os.environ</code> string-soup
SQLAlchemy 2.0	ORM — defines the four tables (<code>database/models.py</code>) and generates SQL for every query	Lets the app swap SQLite for Postgres/MySQL later without rewriting queries
SQLite	The actual database file (<code>sentiment_platform.db</code>)	Zero-config, single-file — appropriate for a project of this scale; no separate DB server to install
python-jose	Encodes/decodes the JWT access tokens issued on login (<code>auth/deps.py</code>)	Standard, well-audited JWT implementation for Python
passlib + bcrypt	Hashes and verifies user passwords before they touch the database	Never store or compare plaintext passwords; bcrypt is the industry-standard slow hash resistant to brute force

AI / ML layer

Technology	Role in this project	Why it was chosen
HuggingFace Transformers 4.44	Loads and runs every pretrained BERT/RoBERTa model used in the pipeline (sentiment, sarcasm, emotion, tone/intent)	The de-facto standard library for using pretrained NLP models without writing custom model code
PyTorch (CPU build)	The actual tensor/inference engine underneath every Transformers model call	Required by Transformers; CPU build keeps the install lightweight since a GPU isn't assumed
scikit-learn	Trains the Logistic Regression head on top of DistilBERT embeddings (<code>modules/sentiment.py</code>) and re-trains it when admins submit corrections	Fast, simple, interpretable classifier — no need for a second neural network just to combine embeddings into a label
langdetect	Step 1 of the pipeline — identifies the input language before anything else runs	Lightweight, no model download, good enough accuracy to route text to the right translation model
sentencepiece / sacremoses	Tokenizers required internally by the MarianMT translation models	Mandatory dependencies of Helsinki-NLP's <code>opus-mt-*</code> models — without them translation fails to load

Social media & document integrations

Technology	Role in this project	Why it was chosen
google-api-python-client	Calls the YouTube Data API v3 to fetch a video's metadata and top comments (<code>modules/url_fetcher.py</code>)	Official Google client library — handles auth and pagination for you
curl_cffi	Fetches Myntra product pages and paginates its internal reviews JSON endpoint (<code>modules/url_fetcher.py::fetch_myntra_reviews</code>)	Myntra has no public reviews API and blocks obvious scraper traffic; <code>curl_cffi</code> impersonates a real Chrome TLS fingerprint so requests aren't rejected the way a plain <code>requests</code> call would be
fpdf2	Generates the branded PDF export (single/bulk/URL analysis reports)	Pure-Python PDF generation with no external binary dependency (unlike <code>wkhtmltopdf</code> -based tools)

Frontend runtime

Technology	Role in this project	Why it was chosen
React 18 + TypeScript 5.2	The entire UI — pages, components, routing	Component model fits a dashboard with many repeated card/table layouts; TypeScript catches API-shape mismatches between frontend and backend at compile time
Vite 5	Dev server (<code>npm run dev</code> , port 5173) and production bundler	Near-instant hot reload during development, much faster cold-start than Webpack-based tooling
Tailwind CSS 3	All component styling, including the theme system (<code>lib/themes.ts</code>)	Utility classes keep styling co-located with markup — no separate CSS files to keep in sync per component
Zustand	Global client state: JWT token, current user, active theme (<code>store/index.ts</code>)	Much less boilerplate than Redux for a store this small; no context-provider wrapping needed
Axios	The HTTP client (<code>lib/api.ts</code>) that talks to the FastAPI backend, attaching the JWT to every request	Built-in interceptors make it easy to inject the auth header and handle 401s in one place
Recharts	Renders the Dashboard's KPI charts, sentiment distribution, trend lines	Declarative, React-native charting API — no manual canvas/SVG wiring
Framer Motion	Page/card transition animations throughout the UI	Simple declarative animation API that works naturally with React component lifecycles
react-dropzone	Drag-and-drop CSV upload on the Bulk Analysis page	Handles file-drop UX edge cases (drag state, validation, multi-file) that are tedious to hand-roll
react-hot-toast	Success/error toast notifications across the app	Lightweight, no provider boilerplate beyond one root component
lucide-react	Icon set used throughout the sidebar, cards, and buttons	Consistent, tree-shakeable icon library that matches the app's clean visual style

The AI models themselves (see §2 for how they connect)

Model	What it actually does here
DistilBERT-multilingual + Logistic Regression	Turns text in 104 possible languages into a 768-dim embedding, then a simple linear classifier maps that embedding to positive/negative/neutral (35% of the final sentiment vote)
Twitter-RoBERTa (sentiment)	A second, independent sentiment vote (65% weight) — trained on 124M tweets, so it understands informal/emoji-heavy comment text better than a Wikipedia-trained model would
Twitter-RoBERTa (irony)	Detects sarcasm; when confident and the sentiment call said "positive," flips it to "negative" (classic sarcasm pattern)
GoEmotions distilRoBERTa	Picks the dominant emotion (joy, anger, sadness, etc.) from 7 classes, then gets corrected if it contradicts the sentiment result
DistilBERT-MNLI	Answers "does this text entail label X?" for 8 tone labels and 8 intent labels — this is what lets tone/intent detection work with zero task-specific training data
Google Translate / MarianMT (Helsinki-NLP)	Translates non-English input into English before the rest of the pipeline runs, so every downstream model only ever has to understand English. Google Translate (no API key, via <code>deep-translator</code>) is tried first; MarianMT is the offline fallback

12. Notable Design Decisions

- **Ensemble weighting (35/65):** Twitter-RoBERTa gets more weight because comments/reviews resemble its tweet training domain more than DistilBERT's Wikipedia pretraining; DistilBERT+LR adds multilingual coverage RoBERTa lacks.

- **Sarcasm-driven sentiment flip:** Runs only after the base sentiment call, and is explicitly skipped when the label came from a human correction — corrections always take precedence over model heuristics.
- **Emotion/sentiment alignment:** Emotion model runs independently of sentiment, so a post-processing step forces the top emotion to stay compatible with the sentiment polarity (no "Joy" on a "Negative" result).
- **Correction cache is the fast path:** Once a text is corrected, `analyze_text()` short-circuits the entire model ensemble for that exact text — cheap and immediate, not a retraining requirement.
- **Corrections persist across restarts:** reloaded from the `correction_entries` table into memory in the FastAPI `lifespan` handler on every boot.
- **Myntra scraping via TLS-impersonated HTTP session**, not a public API (none exists) — `curl_cffi`'s Chrome-120 impersonation avoids the bot-detection that a plain HTTP client would trigger against Myntra's reviews endpoint.
- **Translation has two layers:** Google Translate (via `deep-translator`, no API key) is tried first for broad language coverage, with MarianMT as an offline fallback for a fixed set of languages if Google Translate fails or is unreachable.
- **Default admin account** (`admin` / `admin123`) is auto-seeded on first database initialization (`database/connection.py::_seed_admin`) so the app is usable immediately — the login page also displays this credential as a hint.

13. File-by-File Reference

Backend — entry point & config

File	What it does
<code>backend/main.py</code>	Creates the FastAPI app, registers CORS, mounts every router under <code>/api/v1</code> , and defines the <code>lifespan</code> startup hook: initializes the DB, eagerly loads all ML models (<code>initialize_models()</code>), then reloads any persisted <code>CorrectionEntry</code> rows into the in-memory correction cache so admin fixes survive a restart. Also defines <code>/health</code> , <code>/</code> , and global exception handlers for <code>ValueError</code> (→ 400) and any other exception (→ 500).
<code>backend/config.py</code>	Defines the <code>Settings</code> (Pydantic) class loaded from <code>backend/.env</code> — DB URL, model cache dir, device (<code>auto</code> / <code>cuda</code> / <code>cpu</code>), JWT secret/algorithm/expiry, CORS origins, YouTube API key. Resolves <code>DEVICE=auto</code> to <code>cuda</code> / <code>cpu</code> at import time based on <code>torch.cuda.is_available()</code> .

Backend — auth

File	What it does
<code>auth/deps.py</code>	All auth primitives: <code>hash_password</code> / <code>verify_password</code> (bcrypt via passlib), <code>create_access_token</code> / <code>_decode_token</code> (JWT via python-jose), and three FastAPI dependencies — <code>get_current_user</code> (401 if no/invalid token), <code>get_optional_user</code> (returns <code>None</code> instead of raising, used for anonymous report submission), <code>require_admin</code> (403 if <code>role != "admin"</code>).

Backend — AI/ML pipeline (`modules/`)

File	What it does
<code>modules/pipeline.py</code>	The orchestrator. <code>analyze_text()</code> runs all 8 stages in order and assembles the final result dict. Contains <code>_lookup_correction()</code> (checks the correction cache by original text, translated text, then case-insensitive match) and <code>_align_emotion_with_sentiment()</code> (the <code>_SENTIMENT_EMOTION_COMPAT</code> post-processing step that keeps emotion labels compatible with sentiment polarity). Also exposes <code>initialize_models()</code> , called once at startup to warm up every singleton detector so the first real request isn't slow.
<code>modules/language_detector.py</code>	<code>detect_language()</code> wraps <code>langdetect</code> and maps ISO codes to display names (<code>LANGUAGE_NAMES</code>). <code>is_hinglish()</code> detects Devanagari-script Hindi mixed with Latin characters by character-ratio. <code>is_romanized_hinglish()</code> detects Hindi written in Roman script by matching ≥ 2 common Hindi function words (<code>_ROMANIZED_HINDI_WORDS</code> , a ~90-word list) against $\geq 15\%$ of the text's alphabetic tokens — this is what tells the pipeline to skip MarianMT (which can't handle romanized Hindi) and let multilingual BERT handle it directly.
<code>modules/translation.py</code>	<code>translate_to_english()</code> is the single entry point: returns immediately for English text, otherwise tries <code>_google_translate()</code> (via <code>deep_translator.GoogleTranslator</code> , no key needed) first, then falls back to <code>_marian_translate()</code> for the fixed set of languages in <code>_MARIAN_MODELS</code> . MarianMT models are lazily loaded and cached in <code>_marian_cache</code> on first use per language.
<code>modules/sentiment.py</code>	The core classifier. Defines <code>SEED_DATA</code> (60 hand-written examples spanning formal/informal/emoji text across positive/negative/neutral) and emoji sets (<code>_POSITIVE_EMOJIS</code> / <code>_NEGATIVE_EMOJIS</code>). The <code>BERTLogisticSentimentAnalyzer</code> singleton: <code>_get_embedding()</code> mean-pools DistilBERT's last hidden state; <code>_train_lr()</code> fits the Logistic Regression head on <code>SEED_DATA</code> at startup; <code>analyze()</code> checks the correction cache first, then blends LR probabilities (35%) with the Twitter-RoBERTa pipeline (65%), applies the emoji signal and keyword-signal adjustments from corrections, normalizes, and applies a confidence floor ($<55\% \rightarrow$ neutral) plus a negative/neutral margin guard. <code>apply_single_correction()</code> does an immediate cache-only update; <code>retrain_with_corrections()</code> refits the LR head on seed+correction data and rebuilds both caches; <code>load_online_data()</code> pulls extra labeled samples from HuggingFace's <code>tweet_eval</code> and <code>tyqiangz/multilingual-sentiments</code> datasets to enrich training data on demand.
<code>modules/sarcasm.py</code>	<code>SarcasmDetector</code> singleton wrapping <code>cardiffnlp/twitter-roberta-base-irony</code> . <code>analyze()</code> returns <code>detected=True</code> only when the model's <code>irony</code> label scores $\geq$ <code>SARCASM_THRESHOLD</code> (0.70).
<code>modules/emotion.py</code>	<code>EmotionDetector</code> singleton wrapping the GoEmotions distilRoBERTa model. Maps the model's raw 7 labels to nicer display labels (<code>EMOTION_DISPLAY</code> , e.g. <code>sadness</code> $\rightarrow$ <code>Disappointment</code> , <code>surprise</code> $\rightarrow$ <code>Excitement</code>), taking the max score when two raw labels collapse to the same display label.
<code>modules/tone.py</code> / <code>modules/intent.py</code>	Near-identical <code>ToneDetector</code> / <code>IntentDetector</code> singletons, both wrapping the same <code>typeform/distilbert-base-uncased-mnli</code> zero-shot classifier pipeline but with different candidate label sets (<code>TONE_LABELS</code> vs <code>INTENT_LABELS</code>) and (for tone) natural-language hypothesis templates (<code>TONE_HYPOTHESES</code>) fed into the zero-shot NLI call.
<code>modules/interpretation.py</code>	Pure function <code>generate_interpretation()</code> — no model calls. Stitches together a human-readable paragraph from sentiment/emotion/tone/intent labels using lookup dicts (<code>sent_map</code> , <code>emotion_intent_map</code>) and simple keyword extraction (<code>positive_keywords</code> / <code>negative_keywords</code>) against the translated text.
<code>modules/response_generator.py</code>	Pure function <code>generate_response()</code> — looks up <code>RESPONSE_TEMPLATES</code> keyed by (<code>sentiment</code> , <code>intent</code> , <code>tone</code>) tuples, falling back to partial (<code>sentiment</code> , <code>intent</code>) match, then sentiment-only match, then <code>DEFAULT_RESPONSES</code> . Each match has multiple template variants chosen at random via <code>random.choice</code> .
<code>modules/analytics.py</code>	<code>get_analytics()</code> queries the last N <code>AnalysisRecord</code> rows and computes: label distributions (sentiment/emotion/tone/intent/language) via <code>Counter</code> , average confidence, a

File	What it does
	30-day trend series bucketed by date, a stopwords-filtered word-frequency cloud (<code>top_words</code>), and the 10 most recent reviews. <code>generate_ai_insight()</code> turns those aggregates into a short natural-language summary paragraph (e.g. flags when negative feedback exceeds 50%, or when "complaint" is the dominant intent).
<code>modules/url_fetcher.py</code>	<code>detect_platform()</code> inspects the URL host to route to <code>youtube</code> or <code>mynta</code> (raises <code>ValueError</code> for anything else). <code>fetch_youtube_comments()</code> uses <code>google-api-python-client</code> to pull video metadata and paginate <code>commentThreads.list</code> up to <code>max_comments</code> . <code>fetch_mynta_reviews()</code> uses <code>curl_cffi</code> (Chrome-120 TLS impersonation) to load the product page for its title, extract the numeric product ID from the URL, then paginate Mynta's internal <code>/web/v1/reviews/product/{id}</code> JSON endpoint. <code>fetch_comments()</code> is the single public entry point used by the router.
<code>modules/pdf_generator.py</code>	Three report builders (<code>generate_single_analysis_pdf</code> , <code>generate_bulk_analysis_pdf</code> , <code>generate_url_analysis_pdf</code>) built on a custom <code>_PDF(FPDF)</code> subclass with helpers for colored header bars, progress bars (<code>h_bar</code>), and bordered "cards". Each report renders key metrics, sentiment/emotion distribution bars, highlighted best/worst examples, and a full per-item results table. <code>_safe()</code> sanitizes text to latin-1 (fpdf2's charset) and truncates long strings.

Backend — routers (`routers/` , all mounted under `/api/v1`)

File	What it does
<code>routers/analysis.py</code>	<code>POST /analyze</code> — validates length (≤ 5000 chars), calls <code>analyze_text()</code> , persists the result as a background task (<code>save_analysis_to_db</code> , defined here) so the DB write doesn't block the response.
<code>routers/bulk.py</code>	<code>POST /bulk-analyze</code> — runs up to 100 texts through the pipeline sequentially, persisting each and computing an <code>aggregate</code> summary (distributions + dominants + average confidence). Failed items get a placeholder neutral result instead of aborting the whole batch. <code>POST /upload-csv</code> — parses an uploaded CSV (UTF-8 then latin-1 fallback), heuristically picks a text column (<code>text</code> / <code>review</code> / <code>comment</code> / <code>feedback</code> / <code>content</code> / <code>message</code> , else the first column), and returns up to 500 extracted rows as plain text for the frontend to feed into bulk-analyze.
<code>routers/analytics.py</code>	<code>GET /analytics</code> — thin wrapper that just calls <code>modules/analytics.get_analytics()</code> .
<code>routers/history.py</code>	<code>GET /history</code> (paginated, filterable by sentiment/emotion/language/text search), <code>GET /history/{id}</code> , <code>DELETE /history/{id}</code> , <code>DELETE /history</code> (wipe all). Directly serializes <code>AnalysisRecord</code> rows into the nested response shape the frontend expects.
<code>routers/url_analysis.py</code>	<code>POST /analyze-url</code> — calls <code>fetch_comments()</code> , runs every comment/review through <code>analyze_text()</code> , persists each as an <code>AnalysisRecord</code> with <code>mode="url"</code> , and returns post metadata + per-item results + aggregate stats, mirroring <code>bulk.py</code> 's aggregation logic.
<code>routers/export.py</code>	Three endpoints (<code>/export-pdf</code> , <code>/export-pdf/single</code> , <code>/export-pdf/bulk</code>) that each call the matching <code>pdf_generator</code> function and stream the bytes back with a <code>Content-Disposition: attachment</code> header and a timestamped filename.
<code>routers/auth.py</code>	<code>POST /auth/register</code> (checks username/email uniqueness, hashes password, issues JWT), <code>POST /auth/login</code> (verifies password, 403s if <code>is_active=False</code> , issues JWT), <code>GET /auth/me</code> (returns the current authenticated user).
<code>routers/corrections.py</code>	Admin-only CRUD over <code>CorrectionEntry</code> : <code>POST /corrections</code> (create + immediately push into the in-memory cache via <code>apply_single_correction</code>), <code>GET /corrections</code> , <code>GET /corrections/stats</code> (label breakdown, top keywords, <code>needs_retrain</code> flag at <code>RETRAIN_THRESHOLD=20</code>), <code>DELETE /corrections/{id}</code> , <code>GET /corrections/online-status</code> , <code>POST /corrections/fetch-online</code> (triggers <code>load_online_data</code>), <code>POST /corrections/retrain</code> (triggers <code>retrain_with_corrections</code> on all stored corrections).
<code>routers/reports.py</code>	<code>POST /reports</code> (any user or anonymous — <code>get_optional_user</code>), <code>GET /reports</code> and <code>/reports/stats</code> (admin), <code>PATCH /reports/{id}</code> (admin review: setting <code>status="fixed"</code> with a <code>correct_label</code> auto-creates a <code>CorrectionEntry</code> , retroactively updates every matching <code>AnalysisRecord</code> row so History reflects the fix immediately, and pushes the fix into the live correction cache), <code>DELETE /reports/{id}</code> .
<code>routers/admin.py</code>	Admin-only: <code>GET /admin/stats</code> (user list + counts, SQLite file size vs. a 500MB soft cap via <code>_db_file_size()</code> , record counts across all three tables), <code>PATCH /admin/users/{id}/restrict</code> (toggle <code>is_active</code> , blocked for self), <code>DELETE /admin/users/{id}</code> (blocked for self), <code>DELETE /admin/clear-analysis</code> , <code>DELETE /admin/clear-all</code> .

Backend — database & schemas

File	What it does
database/connection.py	Creates the SQLAlchemy <code>engine</code> / <code>SessionLocal</code> / <code>Base</code> . <code>init_db()</code> creates all tables, runs <code>_migrate_users_table()</code> (adds the <code>is_active</code> column via raw <code>ALTER TABLE</code> if missing — a lightweight SQLite migration since there's no Alembic), and <code>_seed_admin()</code> (creates the default <code>admin</code> / <code>admin123</code> account if no admin exists yet). <code>get_db()</code> is the standard FastAPI session-per-request dependency.
database/models.py	Four SQLAlchemy models: <code>User</code> (auth + role + <code>is_active</code>), <code>UserReport</code> (mis-classification reports with lifecycle status), <code>CorrectionEntry</code> (admin-approved label fixes + keywords), <code>AnalysisRecord</code> (every analysis result, flattened into columns per signal, with <code>mode</code> / <code>batch_id</code> to distinguish single/bulk/url runs).
schemas/analysis.py	Pydantic request/response models mirroring the pipeline's result dict: <code>SentimentResult</code> , <code>SarcasmResult</code> , <code>EmotionResult</code> , <code>ToneResult</code> , <code>IntentResult</code> , <code>AnalysisRequest</code> , <code>BulkAnalysisRequest</code> , <code>SingleAnalysisResponse</code> , <code>BulkAnalysisItem</code> / <code>BulkAnalysisResponse</code> , <code>AnalyticsResponse</code> .
schemas/auth.py	<code>RegisterRequest</code> (username/password/email with length validation), <code>LoginRequest</code> , <code>TokenResponse</code> (access token + embedded <code>UserOut</code>), <code>UserOut</code> .
schemas/corrections.py	<code>CorrectionCreate</code> (label constrained to <code>positive negative neutral</code> via regex), <code>CorrectionResponse</code> , <code>CorrectionStats</code> , <code>RetrainResponse</code> .
schemas/reports.py	<code>ReportCreate</code> , <code>ReportResponse</code> , <code>ReportReview</code> (status constrained to <code>reviewed fixed</code> , optional <code>correct_label</code> / <code>keywords</code>).
schemas/url_analysis.py	<code>URLAnalysisRequest</code> (validates non-empty URL, clamps <code>max_comments</code> to 5–100), <code>PostMetadata</code> , <code>URLAnalysisResponse</code> (reuses <code>BulkAnalysisItem</code> for its <code>items</code> list).

Frontend — app shell & infrastructure

File	What it does
<code>App.tsx</code>	Defines all routes with <code>react-router-dom</code> . <code>RequireAuth</code> redirects to <code>/login</code> if there's no token; <code>RequireAdmin</code> redirects to <code>/dashboard</code> if the user's role isn't <code>admin</code> (client-side guard only — the backend still enforces 401/403 independently). <code>ThemeApplier</code> re-applies the active theme's CSS variables on mount/change. <code>ClearAuthOnStartup</code> logs the user out on every fresh app load (so a stale token doesn't silently persist across browser sessions).
<code>store/index.ts</code>	The single Zustand store (<code>useAppStore</code>). Holds: last analysis results for each mode (single/bulk/URL), analytics cache, loading flags, sidebar collapsed state, active theme ID, the correction-panel's open/prefill/section state, and auth state (<code>user</code> / <code>token</code> , persisted to <code>localStorage</code> under <code>sentimentiq_token</code> / <code>sentimentiq_user</code>). <code>loginUser()</code> / <code>logoutUser()</code> write/clear <code>localStorage</code> alongside in-memory state.
<code>lib/api.ts</code>	The only place that talks to the backend. One <code>axios</code> instance with a request interceptor that attaches <code>Authorization: Bearer <token></code> from <code>localStorage</code> , and a response interceptor that unwraps FastAPI's <code>{detail: "..."} </code> error shape (including from Blob error responses on PDF export calls) into a plain <code>Error</code> . Exports one typed function per backend endpoint (<code>analyzeText</code> , <code>bulkAnalyze</code> , <code>uploadCSV</code> , <code>getAnalytics</code> , <code>getHistory</code> , <code>analyzeURL</code> , the three <code>export*PDF</code> functions plus a shared <code>_downloadPdf</code> helper, all <code>*Correction*</code> / <code>*Report*</code> / <code>*Admin*</code> functions, and auth functions).
<code>lib/themes.ts</code>	Defines 5 color themes (<code>aurora</code> , <code>ocean</code> , <code>rose</code> , <code>emerald</code> , <code>amber</code>) as CSS-variable sets (<code>primary</code> / <code>secondary</code> / <code>accent</code> + 4 "aurora blob" gradient colors used for the background). <code>applyTheme()</code> writes them onto <code>document.documentElement</code> as CSS custom properties; <code>getTheme()</code> / <code>DEFAULT_THEME_ID</code> handle lookup/fallback.
<code>lib/utils.ts</code>	Small pure helpers: <code>cn()</code> (clsx + tailwind-merge class combiner), <code>getSentimentColor</code> / <code>getSentimentBg</code> / <code>getEmotionColor</code> (label→color/class lookups used across every result card), <code>formatConfidence</code> , <code>truncateText</code> , <code>formatDate</code> , <code>capitalize</code> .
<code>types/index.ts</code>	The single source of truth for all API response shapes on the frontend — <code>User</code> , <code>UserReport</code> , <code>ReportStats</code> , <code>SentimentResult</code> / <code>SarcasmResult</code> / <code>EmotionResult</code> / <code>ToneResult</code> / <code>IntentResult</code> , <code>SingleAnalysisResponse</code> , <code>BulkAnalysisItem</code> / <code>BulkAnalysisResponse</code> , <code>AnalyticsData</code> , <code>PostMetadata</code> / <code>URLAnalysisResponse</code> , <code>AdminUserEntry</code> / <code>AdminStats</code> , <code>CorrectionEntry</code> / <code>CorrectionStats</code> , <code>HistoryRecord</code> . Kept in sync by hand with the backend's Pydantic schemas.
<code>components/Layout.tsx</code>	The authenticated app shell — renders the animated "aurora blob" background (using the active theme's CSS variables), the <code>Sidebar</code> , and an <code>Outlet</code> for the current page, shifting its left margin based on whether the sidebar is collapsed.
<code>components/Sidebar.tsx</code>	The main nav rail. Polls <code>/health</code> every 30s to show a live "Backend Connected/Offline" pill, and (for admins) polls <code>/reports/stats</code> every 15s to show a pending-report count badge on the "User Reports" link. Shows/hides the Admin Overview, Training Data, and User Reports links based on <code>user.role</code> . Collapsible via an animated width toggle (Framer Motion).
<code>components/BackendStatus.tsx</code>	A standalone connected/offline pill + manual refresh button around <code>checkHealth()</code> — a smaller, reusable variant of the status indicator embedded in the Sidebar.
<code>components/ReportModal.tsx</code>	The "Report incorrect analysis" dialog shown to any user from an analysis result card. Submits <code>{text, model_label, user_note}</code> to <code>POST /reports</code> and shows a success state before auto-closing.
<code>components/CorrectionPanel.tsx</code>	The large admin-only slide-over panel with two sections (<code>training</code> / <code>reports</code> , switched via the store's <code>correctionPanelSection</code>): the training section lets an admin submit a correction directly, shows correction-count progress toward the retrain threshold, triggers <code>POST /corrections/retrain</code> , and can pull in HuggingFace online datasets via <code>POST /corrections/fetch-online</code> ; the reports section lists pending/reviewed/fixed

File	What it does
	<code>UserReport</code> s with inline "Mark Reviewed" / "Fix & Add to Dataset" actions (the latter requires picking a correct label first).
<code>components/analysis/*.tsx</code>	Small presentational cards, one per pipeline signal, each taking that signal's typed result and rendering a labeled badge/score bar: <code>SentimentCard</code> , <code>EmotionCard</code> , <code>ToneCard</code> , <code>IntentCard</code> , <code>LanguageCard</code> , <code>InterpretationCard</code> , <code>ResponseCard</code> . Reused across <code>SingleAnalysis</code> , <code>BulkAnalysis</code> , and <code>URLAnalysis</code> .
<code>components/ui/*.tsx</code>	Generic design-system primitives shared everywhere: <code>Button</code> (variants + loading spinner + icon slot), <code>Card</code> / <code>CardHeader</code> / <code>CardTitle</code> , <code>Input</code> , <code>Textarea</code> , <code>Select</code> , <code>Badge</code> , <code>Progress</code> , <code>Skeleton</code> (incl. <code>AnalysisCardSkeleton</code> loading placeholder), <code>Toast</code> (react-hot-toast wrapper).

Frontend — pages (`pages/`)

File	What it does
<code>Login.tsx</code>	Combined login/register form (tab-toggled). Calls <code>loginUser</code> / <code>registerUser</code> , stores the returned token+user via the Zustand store, and redirects to <code>/dashboard</code> . Displays the default <code>admin</code> / <code>admin123</code> credential as an on-screen hint.
<code>Dashboard.tsx</code>	Fetches <code>getAnalytics()</code> and renders KPI cards, sentiment/emotion/tone/intent distribution charts (Recharts), a 30-day trend line, a keyword cloud from <code>top_words</code> , the AI-generated insight paragraph, and a recent-reviews list.
<code>SingleAnalysis.tsx</code>	The primary single-text workflow: textarea + example prompts (including Hindi/Hinglish samples) → <code>analyzeText()</code> → renders all seven per-signal cards → supports PDF export (<code>exportSingleAnalysisPDF</code>) and opening <code>ReportModal</code> to flag a wrong result.
<code>BulkAnalysis.tsx</code>	Textarea (newline-separated) or CSV upload (<code>uploadCSV</code>) → <code>bulkAnalyze()</code> → results table with per-row sentiment/emotion/tone/intent, aggregate summary, PDF export (<code>exportBulkAnalysisPDF</code>), and per-row report flagging.
<code>URLAnalysis.tsx</code>	URL input (YouTube or Myntra) → <code>analyzeURL()</code> (long timeout — up to 10 minutes for large comment/review counts) → post metadata header + aggregate summary + per-comment results table, PDF export (<code>exportURLAnalysisPDF</code>).
<code>History.tsx</code>	Paginated, filterable (sentiment/emotion/language/search) table over <code>getHistory()</code> , with per-record delete (<code>deleteHistoryRecord</code>) and a "clear all" action (<code>clearHistory</code>).
<code>Settings.tsx</code>	Theme picker (swaps between the 5 <code>THEMES</code>) and a backend health check panel (<code>checkHealth</code>).
<code>TrainingData.tsx</code>	Admin-only standalone page wrapping the same correction-management functionality as <code>CorrectionPanel</code> 's training section — correction submission, stats, retrain trigger, online-dataset fetch, recent corrections list.
<code>UserReports.tsx</code>	Admin-only standalone page for reviewing user-submitted mis-classification reports (list, stats, filter by status, review/fix/delete actions) — a full-page counterpart to <code>CorrectionPanel</code> 's reports section.
<code>AdminOverview.tsx</code>	Admin-only dashboard: user list with restrict/delete actions (<code>restrictUser</code> , <code>deleteUser</code>), storage usage gauge and record counts (<code>getAdminStats</code>), and destructive data-wipe actions (<code>clearAnalysisRecords</code> , <code>clearAllData</code>).